Modul 3 — DQL (Data Query Language)

Mata pelajaran: Basis Data — RPL **Alat:** MySQL/MariaDB (XAMPP), phpMyAdmin atau terminal **Studi kasus:** Laporan data sekolah (`db_sekolah`) **Prasyarat:** Modul 1 (DDL) dan Modul 2 (DML) selesai.

---

1. Tujuan Pembelajaran

Setelah mempelajari modul ini, kamu mampu:

1. Menampilkan data dengan `SELECT` (semua kolom, kolom tertentu, alias).
2. Menyaring data dengan `WHERE` , `AND` , `OR` , `NOT` , `IN` , `BETWEEN` , `LIKE` , `IS NULL` .
3. Mengurutkan dan membatasi hasil dengan `ORDER BY` , `LIMIT` , `OFFSET` .
4. Meringkas data dengan fungsi agregat, `GROUP BY` , dan `HAVING` .
5. Menggabungkan beberapa tabel dengan `INNER JOIN` dan `LEFT JOIN` .

---

2. Konsep Dasar

DQL (Data Query Language) adalah perintah SQL untuk **mengambil dan menampilkan data**. Perintah utamanya adalah `SELECT` .

> `SELECT` **tidak mengubah** data di tabel. Aman dijalankan berulang kali.

Catatan: sebagian referensi memasukkan `SELECT` ke dalam DML. Pengelompokan DQL terpisah adalah konvensi pembelajaran yang umum.

Urutan penulisan query `SELECT` :

```
SELECT   kolom
FROM     tabel
JOIN     tabel_lain ON kondisi_join
WHERE    kondisi_baris
GROUP BY kolom_kelompok
HAVING   kondisi_kelompok
ORDER BY kolom_urut
LIMIT    jumlah OFFSET lewati;
```

Tidak semua klausa wajib dipakai.

Fungsi tiap klausa:

Klausa	Fungsi
`SELECT`	Memilih kolom atau ekspresi yang ditampilkan
`FROM`	Menentukan tabel sumber
`JOIN ... ON`	Menggabungkan tabel berdasarkan kondisi relasi
`WHERE`	Menyaring baris sebelum dikelompokkan
`GROUP BY`	Mengelompokkan baris untuk perhitungan agregat
`HAVING`	Menyaring hasil kelompok
`ORDER BY`	Mengurutkan hasil
`LIMIT ... OFFSET`	Membatasi jumlah baris dan melewati baris awal

Query ditulis dengan urutan klausa di atas. Secara sederhana, MySQL mencari data dari `FROM`/`JOIN`, menyaring dengan `WHERE`, mengelompokkan, menyaring kelompok dengan `HAVING`, lalu mengurutkan dan membatasi hasil.

---

3. Studi Kasus

Kepala sekolah dan wali kelas meminta berbagai laporan:

- Daftar seluruh siswa dan siswa perempuan saja.
- Siswa dengan nilai di atas 85.
- Jumlah siswa per jurusan dan rata-rata nilai per mapel.
- Siswa yang belum memiliki jurusan atau belum memiliki nilai.
- Rapor sederhana: nama siswa, mapel, dan nilainya.

Semua kebutuhan ini dijawab dengan DQL.

---

4. Persiapan Data

Modul ini memakai data awal dari Modul 2 (3 jurusan, 5 siswa, 3 mapel, 8 nilai). Pastikan hasil di bawah sesuai:

```
USE db_sekolah; -- pilih database yang berisi tabel dan data latihan

SELECT COUNT(*) FROM jurusan;   -- hitung seluruh baris jurusan; hasil acuan: 3
SELECT COUNT(*) FROM siswa;     -- hitung seluruh baris siswa; hasil acuan: 5
SELECT COUNT(*) FROM mapel;     -- hitung seluruh baris mata pelajaran; hasil acuan: 3
SELECT COUNT(*) FROM nilai;     -- hitung seluruh catatan nilai; hasil acuan: 8
```

`USE` memilih database aktif. `COUNT(*)` menghitung semua baris pada tabel yang disebut setelah `FROM`.

Jika data sudah berubah akibat latihan sebelumnya, kamu bisa mengatur ulang **hanya pada database latihan**. Proses ini menghapus seluruh baris pada empat tabel:

```
SET FOREIGN_KEY_CHECKS = 0; -- matikan pemeriksaan relasi sementara agar tabel bisa dikosongkan
TRUNCATE TABLE nilai;       -- kosongkan tabel anak lebih dahulu
TRUNCATE TABLE siswa;       -- kosongkan data siswa
TRUNCATE TABLE mapel;       -- kosongkan data mata pelajaran
TRUNCATE TABLE jurusan;     -- kosongkan tabel induk setelah tabel anak
SET FOREIGN_KEY_CHECKS = 1; -- aktifkan kembali pemeriksaan foreign key
```

Setelah itu, jalankan kembali blok `INSERT` data awal dari Modul 2 bagian 5.1. Pastikan dua perintah `SET FOREIGN_KEY_CHECKS` dijalankan di koneksi yang sama; jangan biarkan pemeriksaan foreign key nonaktif.

Data acuan:

id_siswa	nama	jenis_kelamin	id_jurusan
1	Andi	L	1 (RPL)
2	Siti	P	1 (RPL)
3	Budi	L	2 (DKV)
4	Rina	P	3 (TKJ)
5	Dewi	P	NULL

Tabel `nilai`: (siswa, mapel, nilai) = (1,1,90) (1,2,85) (1,3,88) (2,1,95) (2,2,91) (3,1,78) (3,2,82) (4,1,87). Siswa 5 (Dewi) tidak punya nilai.

---

5. Materi dan Praktik

5.1 `SELECT` dasar

```
-- Semua kolom, semua baris
SELECT * FROM siswa; -- * meminta semua kolom dari tabel siswa

-- Kolom tertentu
SELECT nama, email FROM siswa; -- hasil hanya berisi dua kolom tersebut

-- Alias nama kolom
SELECT nama AS nama_siswa, email AS email_siswa
FROM siswa; -- AS memberi label sementara pada kolom hasil
```

Penjelasan:

- `*` berarti semua kolom.
- Menyebut kolom yang dibutuhkan saja lebih efisien dan jelas.
- `AS` memberi nama sementara pada hasil; nama kolom asli tidak berubah.

5.2 `DISTINCT` — menghilangkan duplikat hasil

```
SELECT DISTINCT id_jurusan FROM siswa; -- tampilkan tiap nilai jurusan satu kali

SELECT COUNT(DISTINCT id_jurusan) AS jumlah_jurusan
FROM siswa; -- hitung ID jurusan yang berbeda; NULL tidak dihitung
```

Penjelasan: `DISTINCT` membuang duplikat hanya dari hasil query, bukan dari data di tabel. `COUNT(DISTINCT ...)` tidak menghitung `NULL`, sehingga untuk data acuan jumlah jurusan yang memiliki siswa adalah 3.

5.3 `WHERE` — menyaring baris

```
SELECT * FROM siswa WHERE jenis_kelamin = 'P'; -- tampilkan baris dengan jenis_kelamin P

SELECT * FROM nilai WHERE nilai_angka >= 85; -- >= berarti lebih besar atau sama dengan 85

SELECT * FROM nilai WHERE nilai_angka <> 90; -- <> berarti tidak sama dengan 90
```

Operator pembanding

Operator	Arti
`=`	Sama dengan
`<>` atau `!=`	Tidak sama dengan
`>` `<`	Lebih besar / lebih kecil
`>=` `<=`	Lebih besar atau sama / lebih kecil atau sama

`SELECT` menentukan kolom hasil, `FROM` menentukan tabel sumber, dan `WHERE` menentukan baris yang lolos. Hanya baris dengan kondisi benar yang tampil.

5.4 `AND`, `OR`, `NOT`

```
-- Kedua kondisi harus benar
SELECT * FROM nilai
WHERE nilai_angka >= 80 AND semester = 1; -- harus memenuhi kedua kondisi

-- Salah satu cukup benar
SELECT * FROM siswa
WHERE id_jurusan = 1 OR id_jurusan = 2; -- cukup cocok dengan salah satu ID jurusan

-- Membalik kondisi
SELECT * FROM siswa
WHERE NOT jenis_kelamin = 'L'; -- pilih baris yang jenis kelaminnya bukan L

-- Gabungan: gunakan tanda kurung
SELECT * FROM nilai
WHERE semester = 1
  AND (nilai_angka >= 90 OR nilai_angka < 80); -- semester 1 dan salah satu syarat nilai
```

`AND` mengharuskan semua kondisi terkait benar, sedangkan `OR` cukup salah satu benar. `NOT` membalik hasil kondisi. `AND` lebih dulu dievaluasi daripada `OR`; tanda kurung membuat maksud query jelas.

5.5 `IN`, `BETWEEN`, `LIKE`

```
-- IN: cocok dengan salah satu nilai dalam daftar
SELECT * FROM siswa WHERE id_jurusan IN (1, 3); -- setara dengan id_jurusan = 1 OR id_jurusan = 3

-- BETWEEN: rentang, batas bawah dan atas ikut
SELECT * FROM nilai WHERE nilai_angka BETWEEN 80 AND 90; -- termasuk nilai 80 dan 90

-- LIKE: pencarian pola
SELECT * FROM siswa WHERE nama LIKE 'A%';    -- nama diawali huruf A
SELECT * FROM siswa WHERE nama LIKE '%i';    -- nama diakhiri huruf i
SELECT * FROM siswa WHERE nama LIKE '%nd%';  -- nama mengandung rangkaian "nd"
SELECT * FROM siswa WHERE nama LIKE '_ndi';  -- tepat satu karakter diikuti "ndi"
```

Penjelasan:

- `%` = nol atau lebih karakter; `_` = tepat satu karakter.
- Sensitivitas huruf besar/kecil bergantung pada *collation*.

5.6 `IS NULL` dan `IS NOT NULL`

```
-- Siswa yang belum punya jurusan
SELECT * FROM siswa WHERE id_jurusan IS NULL; -- nilai jurusan belum tersedia

-- Siswa yang sudah punya jurusan
SELECT * FROM siswa WHERE id_jurusan IS NOT NULL; -- nilai jurusan tersedia
```

`NULL` berarti "tidak ada/tidak diketahui", bukan angka nol atau teks kosong. Perbandingan biasa seperti `= NULL` tidak menghasilkan kondisi benar; gunakan `IS NULL` atau `IS NOT NULL`.

5.7 `ORDER BY`, `LIMIT`, `OFFSET`

```
-- Urut nama A-Z
SELECT * FROM siswa ORDER BY nama ASC; -- urut berdasarkan nama dari A ke Z

-- Nilai tertinggi ke terendah
SELECT * FROM nilai ORDER BY nilai_angka DESC; -- DESC mengurutkan dari terbesar ke terkecil

-- Urut bertingkat
SELECT * FROM siswa ORDER BY id_jurusan ASC, nama ASC; -- urut jurusan, lalu nama dalam tiap jurusan

-- 3 data pertama
SELECT * FROM siswa ORDER BY id_siswa LIMIT 3; -- urutkan dahulu, lalu ambil maksimal 3 baris

-- Halaman ke-2 (3 data per halaman)
SELECT * FROM siswa ORDER BY id_siswa LIMIT 3 OFFSET 3; -- lewati 3 baris pertama, ambil 3 berikutnya
```

Penjelasan:

- `ASC` menaik (default), `DESC` menurun.
- `LIMIT` membatasi jumlah baris; `OFFSET` melewati sejumlah baris awal. Kombinasi keduanya dipakai untuk *pagination*.
- Selalu pakai `ORDER BY` bersama `LIMIT` agar hasil konsisten.

Kasus: "Tampilkan 3 nilai tertinggi."

```
SELECT * FROM nilai ORDER BY nilai_angka DESC LIMIT 3; -- tampilkan tiga nilai paling tinggi
```

5.8 Fungsi agregat

```
SELECT COUNT(*) AS total_siswa FROM siswa; -- hitung baris; AS memberi nama total_siswa pada hasil

SELECT
    AVG(nilai_angka) AS rata_rata, -- hitung nilai rata-rata
    MIN(nilai_angka) AS terendah,  -- cari nilai paling kecil
    MAX(nilai_angka) AS tertinggi, -- cari nilai paling besar
    SUM(nilai_angka) AS total      -- jumlahkan semua nilai
FROM nilai;                        -- sumber perhitungan adalah tabel nilai
```

Fungsi	Kegunaan
`COUNT()`	Menghitung jumlah baris/nilai
`SUM()`	Menjumlahkan
`AVG()`	Rata-rata
`MIN()` / `MAX()`	Terkecil / terbesar

Penjelasan: `COUNT(*)` menghitung semua baris, sedangkan `COUNT(kolom)` hanya menghitung yang tidak `NULL`. Contoh: `COUNT(id_jurusan)` pada tabel siswa bernilai 4, bukan 5.

5.9 `GROUP BY` dan `HAVING`

```
-- Jumlah siswa per jurusan
SELECT id_jurusan, COUNT(*) AS jumlah_siswa
FROM siswa
GROUP BY id_jurusan; -- buat satu kelompok untuk tiap nilai id_jurusan

-- Rata-rata nilai per mapel
SELECT id_mapel, AVG(nilai_angka) AS rata_rata
FROM nilai
GROUP BY id_mapel; -- hitung rata-rata secara terpisah untuk tiap mapel

-- Hanya mapel dengan rata-rata >= 85
SELECT id_mapel, AVG(nilai_angka) AS rata_rata
FROM nilai
GROUP BY id_mapel
HAVING AVG(nilai_angka) >= 85; -- pilih kelompok yang rata-ratanya minimal 85
```

Penjelasan:

- `GROUP BY` mengelompokkan baris yang nilainya sama, lalu fungsi agregat menghitung per kelompok.
- `WHERE` menyaring **baris sebelum** pengelompokan; `HAVING` menyaring **kelompok setelah** pengelompokan.

Kasus gabungan: "Mapel dengan nilai di atas 80 saja yang dihitung rata-ratanya, tampilkan yang rata-ratanya minimal 85."

```
SELECT id_mapel, AVG(nilai_angka) AS rata_rata
FROM nilai
WHERE nilai_angka > 80          -- pilih baris nilai di atas 80 sebelum pengelompokan
GROUP BY id_mapel               -- kelompokkan baris tersisa per mapel
HAVING AVG(nilai_angka) >= 85   -- pilih kelompok dengan rata-rata minimal 85
ORDER BY rata_rata DESC;        -- urutkan rata-rata dari tertinggi
```

5.10 `JOIN` — menggabungkan tabel

Tabel `siswa` hanya menyimpan `id_jurusan`. Untuk menampilkan **nama** jurusan, gabungkan dengan tabel `jurusan` berdasarkan kolom yang berhubungan.

a. `INNER JOIN` — hanya yang cocok di kedua tabel

```
SELECT s.id_siswa, s.nama, j.nama_jurusan
FROM siswa AS s
INNER JOIN jurusan AS j
    ON s.id_jurusan = j.id_jurusan; -- cocokkan ID jurusan yang sama di kedua tabel
```

`s` dan `j` adalah alias agar nama tabel lebih ringkas. `INNER JOIN` hanya menampilkan baris yang memiliki pasangan berdasarkan kondisi `ON`. Hasilnya 4 baris; Dewi tidak muncul karena `id_jurusan`-nya `NULL` sehingga tidak ada pasangan.

b. `LEFT JOIN` — semua data tabel kiri

```
SELECT s.id_siswa, s.nama, j.nama_jurusan
FROM siswa AS s
LEFT JOIN jurusan AS j
    ON s.id_jurusan = j.id_jurusan; -- pertahankan semua siswa meskipun tidak punya pasangan
```

Semua baris di tabel kiri (`siswa`) tetap tampil. Jika tidak ada pasangan di `jurusan`, kolom dari tabel kanan bernilai `NULL`. Hasilnya 5 baris, termasuk Dewi.

c. Mencari data tanpa pasangan

```
-- Siswa belum punya jurusan
SELECT s.id_siswa, s.nama
FROM siswa AS s
LEFT JOIN jurusan AS j ON s.id_jurusan = j.id_jurusan
WHERE j.id_jurusan IS NULL; -- NULL di sisi kanan berarti siswa tidak punya pasangan jurusan

-- Siswa belum punya nilai
SELECT s.id_siswa, s.nama
FROM siswa AS s
LEFT JOIN nilai AS n ON s.id_siswa = n.id_siswa
WHERE n.id_nilai IS NULL; -- cari siswa yang tidak memiliki catatan nilai
```

Penjelasan: pola `LEFT JOIN ... WHERE kolom_tabel_kanan IS NULL` sangat berguna untuk menemukan data yang belum berelasi.

d. `RIGHT JOIN` — semua data tabel kanan

```
SELECT s.nama, j.nama_jurusan
FROM siswa AS s
RIGHT JOIN jurusan AS j
    ON s.id_jurusan = j.id_jurusan; -- semua jurusan (tabel di sisi kanan) tetap tampil
```

Penjelasan: semua jurusan tampil walau belum punya siswa. `RIGHT JOIN` bisa ditulis ulang sebagai `LEFT JOIN` dengan menukar urutan tabel.

e. `JOIN` tiga tabel — rapor sederhana

```
SELECT s.nama, m.nama_mapel, n.nilai_angka, n.semester
FROM siswa AS s
INNER JOIN nilai AS n ON s.id_siswa = n.id_siswa -- hubungkan siswa dengan catatan nilainya
INNER JOIN mapel AS m ON n.id_mapel = m.id_mapel -- ambil nama mapel untuk setiap nilai
ORDER BY s.nama, m.nama_mapel;                    -- urutkan berdasarkan nama siswa lalu mapel
```

Penjelasan: `nilai` menjadi jembatan antara `siswa` dan `mapel`. Hasil: 8 baris.

f. `JOIN` dengan agregasi — rata-rata per siswa

```
SELECT
    s.id_siswa,
    s.nama,
    AVG(n.nilai_angka) AS rata_rata,
    COUNT(n.id_nilai)  AS jumlah_nilai
FROM siswa AS s
LEFT JOIN nilai AS n ON s.id_siswa = n.id_siswa
GROUP BY s.id_siswa, s.nama -- satu hasil agregat per siswa
ORDER BY rata_rata DESC;    -- urutkan rata-rata tertinggi lebih dahulu
```

Penjelasan: `LEFT JOIN` menjaga Dewi tetap tampil (rata-rata `NULL`, jumlah nilai 0). Gunakan `COUNT(n.id_nilai)`, bukan `COUNT(*)`, agar siswa tanpa nilai dihitung 0.

JOIN	Baris yang tampil
`INNER JOIN`	Hanya yang cocok di kedua tabel
`LEFT JOIN`	Semua tabel kiri + pasangan (jika ada)
`RIGHT JOIN`	Semua tabel kanan + pasangan (jika ada)

---

6. Rangkuman

Klausa	Fungsi
`SELECT ... FROM`	Memilih kolom dan tabel
`WHERE`	Menyaring baris
`ORDER BY`	Mengurutkan
`LIMIT` / `OFFSET`	Membatasi dan melewati baris
`GROUP BY`	Mengelompokkan
`HAVING`	Menyaring kelompok
`JOIN`	Menggabungkan tabel
`COUNT/SUM/AVG/MIN/MAX`	Ringkasan data

7. Latihan

Gunakan `db_sekolah` dengan data acuan.

Dasar

1. Tampilkan nama dan email semua siswa.
2. Tampilkan siswa laki-laki.
3. Tampilkan siswa yang namanya mengandung huruf "i".
4. Tampilkan siswa yang lahir pada tahun 2008.

Menengah

5. Tampilkan semua nilai antara 80 dan 90, urut dari tertinggi.
6. Tampilkan 3 nilai tertinggi.
7. Hitung jumlah siswa per jenis kelamin.
8. Tampilkan mapel dengan rata-rata nilai minimal 85.

JOIN

9. Tampilkan nama siswa beserta nama jurusannya (siswa tanpa jurusan tetap tampil).
10. Tampilkan rapor: nama siswa, nama mapel, dan nilai.
11. Tampilkan jurusan yang belum memiliki siswa.
12. Tampilkan rata-rata nilai tiap siswa, urut dari tertinggi.

Pertanyaan diskusi: mengapa `INNER JOIN` pada soal 9 menghilangkan Dewi, sedangkan `LEFT JOIN` tidak?

8. Kesalahan Umum

Masalah	Penyebab	Solusi
`= NULL` tidak menemukan apa pun	`NULL` bukan nilai biasa	Pakai `IS NULL`
`Column ... is ambiguous`	Nama kolom sama di dua tabel	Awali dengan alias tabel (`s.id_siswa`)
`GROUP BY` error (`ONLY_FULL_GROUP_BY`)	Kolom di `SELECT` tidak ada di `GROUP BY` dan bukan agregat	Tambahkan ke `GROUP BY` atau gunakan agregat
Siswa hilang dari hasil `JOIN`	Memakai `INNER JOIN`	Pakai `LEFT JOIN`
Hasil `LIMIT` berubah-ubah	Tanpa `ORDER BY`	Selalu sertakan `ORDER BY`
Rata-rata salah karena `WHERE`	`WHERE` menyaring sebelum agregasi	Pakai `HAVING` untuk kondisi pada hasil agregat